

ORACLE SQL

Final-Half Exam Reviewer

Joins through time zones - capabilities, limitations, comparisons, and exam-style questions

Continuation: This reviewer continues directly from <File>Oracle SQL First-Half Exam Reviewer</File>, whose boundary ends after aggregate reporting. The final half begins with multiple-table retrieval and covers every remaining syllabus area.

Method: Every section explains what the syntax can do, when it is valid, when it fails, how it differs from similar syntax, and how Oracle certification questions turn those rules into alternatives.

How to Solve Final-Half Questions

- For joins, list matching combinations and identify the preserved side.
- For subqueries, identify returned columns, possible row count, correlation, and NULL possibility.
- For set operators, compare column count, compatible type groups, duplicate behavior, and final ORDER BY.
- For DML, trace target rows, constraints, statement atomicity, savepoints, and transaction end.
- For DDL, distinguish row removal from object removal and remember implicit transaction effects.
- For privileges, identify whether the requested action is system-level or object-level.
- For time zones, identify exact return data type, session-zone effect, and interval family.

Module 7 - Displaying Data from Multiple Tables

7.1 Inner joins and multiple-table retrieval

A join combines rows from two or more row sources when the join condition evaluates TRUE.

Capabilities

- ANSI JOIN separates relationship conditions in ON from filters in WHERE.
- Equijoins use equality.
- Qualify same-named columns with table aliases.
- With several tables, every required relationship must be connected.

Syntax and example

```
SELECT e.employee_name,d.department_name
FROM employees e
JOIN departments d ON e.department_id=d.department_id
WHERE d.location_id=1700;
```

Limitations and valid-use rules

- Only matching row combinations survive an inner join.
- A parent row can repeat when several child rows match.
- Missing one join condition can multiply rows unexpectedly.
- Column aliases do not replace table aliases.

Comparison with similar syntax

Feature	Best use	Key difference
INNER JOIN	Matching rows only	Outer join preservation
ON	General join condition	WHERE-only filtering
USING	Same-named join column	Qualifying the USING column with a table alias

Exam construction pattern: Choices frequently differ by where a filter is placed or by an omitted relationship.

7.2 Self-joins

A self-join uses one physical table as two logical roles, such as employee and manager.

Capabilities

- Aliases distinguish roles.
- The relationship may be hierarchical but SQL still joins two row instances.
- LEFT JOIN keeps employees without managers.

Syntax and example

```
SELECT e.employee_name employee,
       m.employee_name manager
FROM employees e
LEFT JOIN employees m ON e.manager_id=m.employee_id;
```

Limitations and valid-use rules

- Aliases are essential for clarity and ambiguity avoidance.

- An inner self-join omits top-level employees with NULL manager_id.
- A self-join does not duplicate the table physically.

Comparison with similar syntax

Feature	Best use	Key difference
Self-join	Compare rows in one table	Two stored copies
LEFT self-join	Keep employees without managers	Inner self-join

Exam construction pattern: The exam may ask whether KING with NULL manager appears.

7.3 Non-equijoins and outer joins

A non-equijoin uses a range or inequality. Outer joins retain unmatched rows from one or both sides.

Capabilities

- BETWEEN can map salary to a grade range.
- LEFT preserves left rows; RIGHT preserves right; FULL preserves both.
- Unmatched side columns become NULL.

Syntax and example

```
SELECT e.employee_name,g.grade
FROM employees e JOIN salary_grade g
ON e.salary BETWEEN g.low_salary AND g.high_salary;
```

```
SELECT d.department_name,e.employee_name
FROM departments d LEFT JOIN employees e
ON d.department_id=e.department_id;
```

Limitations and valid-use rules

- Overlapping ranges can match one employee to multiple grades.
- A WHERE predicate on the nullable side can accidentally remove outer-joined rows.
- FULL OUTER JOIN includes matches plus unmatched rows from both sides.

Comparison with similar syntax

Feature	Best use	Key difference
LEFT JOIN	Preserve left	FULL preserves both
Non-equijoin	Range/inequality matching	Equijoin =
CROSS JOIN	All combinations	FULL OUTER JOIN

Exam construction pattern: A classic trap moves e.status=ACTIVE from ON to WHERE and changes preservation.

7.4 Cartesian products

A Cartesian product pairs every row of one source with every row of another.

Capabilities

- CROSS JOIN requests it explicitly.
- Missing join conditions can create it accidentally.
- Row count is product of source row counts after applicable independent filters.

Syntax and example

```
SELECT e.employee_name,d.department_name  
FROM employees e CROSS JOIN departments d;
```

Limitations and valid-use rules

- Large inputs cause rapid row growth.
- A Cartesian product is not the same as a full outer join.
- DISTINCT may hide repeated outputs but does not repair missing relationship logic.

Comparison with similar syntax

Feature	Best use	Key difference
CROSS JOIN	Every pair	INNER matching
FULL OUTER	Matches plus unmatched sides	Every possible pair

Exam construction pattern: Given 4 and 3 rows, answer 12, not 7.

Module 8 - Using Subqueries to Solve Queries

8.1 Single-row subqueries

A single-row subquery supplies at most one row for a scalar comparison.

Capabilities

- Use =, <>, >, <, >=, <=.
- Aggregate without GROUP BY commonly produces one scalar row.
- A no-row scalar subquery behaves like NULL in the comparison.

Syntax and example

```
SELECT employee_name,salary
FROM employees
WHERE salary>(SELECT AVG(salary) FROM employees);
```

Limitations and valid-use rules

- More than one returned row raises a single-row subquery error.
- The subquery must return one comparison column.
- NULL comparison can yield UNKNOWN.

Comparison with similar syntax

Feature	Best use	Key difference
= subquery	One scalar row	IN multi-row set
AVG subquery	Guaranteed aggregate scalar row	Grouped AVG returning several rows

Exam construction pattern: Determine cardinality before choosing the operator.

8.2 Multiple-row subqueries

Multiple-row subqueries return a set used by IN, ANY, ALL, or EXISTS.

Capabilities

- IN means equal to any returned value.
- > ANY means greater than at least one value.
- > ALL means greater than every value.
- EXISTS tests whether at least one row exists.

Syntax and example

```
SELECT employee_name FROM employees
WHERE department_id IN
(SELECT department_id FROM departments WHERE location_id=1700);
```

Limitations and valid-use rules

- IN needs one comparison column.
- NOT IN with a NULL in the set can make the condition UNKNOWN.
- ANY and ALL require comparison operators.
- EXISTS ignores selected values and checks row existence.

Comparison with similar syntax

Feature	Best use	Key difference
IN	Membership	Scalar =

ANY	At least one comparison	ALL every comparison
EXISTS	Row existence	Returned scalar value

Exam construction pattern: NULL inside NOT IN is a high-frequency trap.

8.3 Correlated UPDATE and DELETE

A correlated subquery references the current row of the outer DML statement.

Capabilities

- UPDATE can derive a value from related rows.
- DELETE can remove rows for which a related row exists or does not exist.
- Aliases clarify outer and inner references.

Syntax and example

```
UPDATE employees e
SET salary=(SELECT j.standard_salary FROM jobs j
            WHERE j.job_id=e.job_id)
WHERE EXISTS (SELECT 1 FROM jobs j WHERE j.job_id=e.job_id);
```

```
DELETE FROM employees e
WHERE NOT EXISTS
(SELECT 1 FROM departments d WHERE d.department_id=e.department_id);
```

Limitations and valid-use rules

- A scalar SET subquery must return at most one row per updated row.
- Without a protective WHERE/EXISTS, no-row SET subquery may assign NULL.
- Correlated logic is conceptual; the optimizer may transform execution.

Comparison with similar syntax

Feature	Best use	Key difference
Correlated subquery	Depends on outer row	Independent scalar subquery
EXISTS	Protect/choose related rows	Select returned values

Exam construction pattern: Questions inspect what happens when the correlated subquery returns zero or multiple rows.

Module 9 - Set Operators

9.1 Matching SELECT statements

Set operators combine result sets vertically.

Capabilities

- Each component returns the same number of columns.
- Corresponding positions require compatible data-type groups.
- Final headings derive from the first SELECT.

Syntax and example

```
SELECT employee_id,employee_name FROM current_emp
UNION
SELECT contractor_id,contractor_name FROM contractors;
```

Limitations and valid-use rules

- Column names need not match.
- A NUMBER position cannot simply match arbitrary DATE text without valid compatibility/conversion.
- ORDER BY belongs at the end of the compound query.

Comparison with similar syntax

Feature	Best use	Key difference
UNION	Combine unique rows	JOIN horizontal combination
JOIN	Combine columns by relationship	Set vertical stacking

Exam construction pattern: The exam may show matching names but incompatible type positions.

9.2 UNION, UNION ALL, INTERSECT, MINUS

Each set operator has distinct duplicate and inclusion behavior.

Capabilities

- UNION removes duplicate complete rows.
- UNION ALL retains duplicates.
- INTERSECT returns rows common to both.
- MINUS returns distinct rows in first result and absent from second.

Syntax and example

```
SELECT department_id FROM active_dept
MINUS
SELECT department_id FROM closed_dept
ORDER BY department_id;
```

Limitations and valid-use rules

- Set operators compare complete rows, not one selected field independently.
- MINUS direction matters.
- UNION duplicate removal may require extra work compared with UNION ALL.

Comparison with similar syntax

Feature	Best use	Key difference
---------	----------	----------------

UNION	Unique combined rows	UNION ALL duplicates kept
INTERSECT	Common rows	MINUS first-only
MINUS	First minus second	Symmetric difference

Exam construction pattern: Choices often reverse MINUS direction or claim UNION ALL removes duplicates.

9.3 ORDER BY in set operations

One final ORDER BY sorts the complete compound result.

Capabilities

- Use first-query output names, aliases, or valid positions.
- The final ORDER BY applies after set combination.

Syntax and example

```
SELECT employee_id id,employee_name name FROM employees
UNION ALL
SELECT contractor_id,contractor_name FROM contractors
ORDER BY name;
```

Limitations and valid-use rules

- An alias introduced only in a later component may not be the output heading.
- Independent ORDER BY clauses are not placed after each ordinary component.
- Sorting does not change duplicate-handling behavior.

Comparison with similar syntax

Feature	Best use	Key difference
Final ORDER BY	Sort combined result	Sort each component independently
Position ORDER BY	Reference output position	Stable under SELECT-list edits

Exam construction pattern: Look for an alias that exists only in the second SELECT.

Module 10 - Managing Tables with DML

10.1 INSERT, UPDATE, DELETE

DML changes rows and participates in a transaction.

Capabilities

- INSERT supplies rows. UPDATE changes qualifying rows. DELETE removes qualifying rows.
- Subqueries may supply values or identify targets.
- WHERE scope must be verified before UPDATE or DELETE.

Syntax and example

```
INSERT INTO departments(department_id,department_name)
VALUES(50,'SUPPORT');
UPDATE employees SET salary=salary*1.05 WHERE department_id=20;
DELETE FROM employees WHERE status='INACTIVE';
```

Limitations and valid-use rules

- Omitted required columns cause errors unless defaults apply.
- UPDATE/DELETE without WHERE targets all rows.
- Constraint checks can reject DML.
- A failed statement is atomic.

Comparison with similar syntax

Feature	Best use	Key difference
DELETE	Row removal, table remains	DROP object removal
UPDATE	Change rows	ALTER TABLE structure
INSERT	Add rows	CREATE TABLE

Exam construction pattern: Options often differ by omitted column list, NULL constraints, or missing WHERE.

10.2 Transactions

A transaction groups logical work until commit or rollback.

Capabilities

- COMMIT makes work permanent and releases transaction locks.
- ROLLBACK reverses uncommitted work.
- SAVEPOINT marks a partial rollback location.
- ROLLBACK TO reverses work after the marker without ending the whole transaction.

Syntax and example

```
INSERT INTO dept VALUES(10,'SALES');
SAVEPOINT after_insert;
UPDATE dept SET dept_name='GLOBAL SALES' WHERE dept_id=10;
ROLLBACK TO after_insert;
COMMIT;
```

Limitations and valid-use rules

- DDL can create implicit commit boundaries.
- COMMIT erases savepoints.
- Sequence values are not restored by rollback.
- Other sessions normally do not see uncommitted changes under standard read consistency.

Comparison with similar syntax

Feature	Best use	Key difference
COMMIT	Make all current transaction work permanent	SAVEPOINT
ROLLBACK	Undo uncommitted transaction	ROLLBACK TO partial
SAVEPOINT	Marker	Permanent save

Exam construction pattern: Trace each statement chronologically and mark what occurs before/after savepoint.

10.3 Multitable INSERT

Multitable INSERT routes rows from one driving query into several target clauses.

Capabilities

- INSERT ALL executes all qualifying INTO clauses.
- Conditional INSERT ALL can target several tables.
- INSERT FIRST uses only the first matching WHEN branch per source row.

Syntax and example

```
INSERT ALL
  WHEN amount<1000 THEN INTO small_orders VALUES(order_id,amount)
  WHEN amount>=1000 THEN INTO large_orders VALUES(order_id,amount)
SELECT order_id,amount FROM orders_stage;
```

Limitations and valid-use rules

- It is driven by a subquery, even when DUAL supplies one row.
- Constraints on any target can fail the statement.
- Sequence behavior in multitable inserts is an exam trap; trace generated values and target keys carefully.

Comparison with similar syntax

Feature	Best use	Key difference
INSERT ALL	All matching branches	INSERT FIRST first match
Single-table INSERT	One target	Multitable routing

Exam construction pattern: Determine how many source rows and how many branches qualify.

10.4 MERGE

MERGE synchronizes a target with a source according to a match condition.

Capabilities

- WHEN MATCHED commonly updates.
- WHEN NOT MATCHED commonly inserts.
- Optional delete behavior is tied to MERGE restrictions and matched updates.

Syntax and example

```
MERGE INTO inventory t
USING shipment s ON(t.product_id=s.product_id)
WHEN MATCHED THEN UPDATE SET t.qty=t.qty+s.qty
```

```
WHEN NOT MATCHED THEN INSERT(product_id,qty)
VALUES(s.product_id,s.qty);
```

Limitations and valid-use rules

- Source duplicates matching one target can create unstable/multiple-update issues.
- MERGE is transactional DML, not automatic per-row commit.
- The ON condition determines matched status and has update restrictions.

Comparison with similar syntax

Feature	Best use	Key difference
MERGE	Conditional insert/update	Independent unconditional INSERT
ON	Match classification	Final output sorting

Exam construction pattern: Questions ask which branch runs for matched versus unmatched rows.

Module 11 - Indexes, Synonyms, and Sequences

11.1 Indexes

An index is a schema object that can provide faster access to table rows.

Capabilities

- Single-column, composite, unique, and function-based indexes are common forms.
- The optimizer decides whether to use an index.
- Indexes can support constraints but are not identical to constraints.

Syntax and example

```
CREATE INDEX emp_dept_i ON employees(department_id);
CREATE INDEX emp_upper_name_i ON employees(UPPER(last_name));
```

Limitations and valid-use rules

- Indexes require storage and DML maintenance.
- Low selectivity or large result percentages may favor a full scan.
- Composite column order matters.
- A normal index on LAST_NAME does not exactly match UPPER(LAST_NAME).

Comparison with similar syntax

Feature	Best use	Key difference
Unique index	Unique key entries	Primary-key business designation
Function-based index	Indexed expression	Normal raw-column index
Composite index	Ordered column combination	Independent indexes

Exam construction pattern: “An available index is always used” is false.

11.2 Synonyms

A synonym is an alternate name for another object.

Capabilities

- Private synonyms belong to a schema.
- Public synonyms are broadly visible for name resolution.
- They simplify qualified names or isolate applications from object-name changes.

Syntax and example

```
CREATE SYNONYM emp FOR hr.employees;
SELECT * FROM emp;
```

Limitations and valid-use rules

- A synonym does not grant privileges.
- The target object must exist or validity/access must be available when used.
- Public visibility is not equivalent to public data authorization.

Comparison with similar syntax

Feature	Best use	Key difference
---------	----------	----------------

Synonym	Alternate name	Privilege grant
View	Stored query abstraction	Name alias only

Exam construction pattern: A user with a synonym but no SELECT privilege still cannot query the target.

11.3 Sequences

A sequence generates numeric values independently of a table.

Capabilities

- NEXTVAL advances and returns a value.
- CURRVAL returns the session current value after NEXTVAL.
- CACHE improves performance but can increase visible gaps.

Syntax and example

```
CREATE SEQUENCE emp_seq START WITH 100 INCREMENT BY 1 NOCACHE;
INSERT INTO employees(employee_id,employee_name)
VALUES(emp_seq.NEXTVAL,'ANA');
```

Limitations and valid-use rules

- Sequence values can have gaps.
- Rollback does not restore a consumed value.
- CURRVAL is undefined in a new session until NEXTVAL is used.
- A sequence does not automatically populate a column unless referenced or connected through another feature.

Comparison with similar syntax

Feature	Best use	Key difference
NEXTVAL	Advance and return	CURRVAL current session value
Sequence	Generate values	Identity/trigger automatic behavior

Exam construction pattern: Watch session state, gaps, and rollback assumptions.

Module 12 - DDL, Tables, Relationships, and Constraints

12.1 Tables, columns, and data types

CREATE TABLE defines columns, data types, defaults, and constraints.

Capabilities

- NUMBER(p,s), VARCHAR2, CHAR, DATE, TIMESTAMP, CLOB, BLOB.
- ALTER TABLE changes structure.
- Defaults apply according to insert/default syntax and column definition.

Syntax and example

```
CREATE TABLE project(
  project_id NUMBER(8) CONSTRAINT project_pk PRIMARY KEY,
  project_name VARCHAR2(100) NOT NULL,
  start_date DATE DEFAULT SYSDATE,
  budget NUMBER(12,2));
```

Limitations and valid-use rules

- NUMBER(6,2) permits four integer digits and two fractional digits.
- VARCHAR2 is variable-length; CHAR is fixed-length/padded semantics.
- A default does not generally replace every explicit NULL unless definition features specify that behavior.

Comparison with similar syntax

Feature	Best use	Key difference
NUMBER(p,s)	Precision and scale	String length
VARCHAR2(n)	Variable text maximum	CHAR fixed-length behavior
DATE	Date/time to seconds	Time-zone-aware timestamp

Exam construction pattern: The exam may test whether a literal fits precision, length, and NOT NULL rules.

12.2 Dropping columns, UNUSED, and TRUNCATE

DDL can remove schema components or all table rows.

Capabilities

- DROP COLUMN removes the column definition and data.
- SET UNUSED makes a column inaccessible quickly.
- DROP UNUSED COLUMNS completes removal.
- TRUNCATE removes all rows while retaining the table object.

Syntax and example

```
ALTER TABLE project SET UNUSED(notes);
ALTER TABLE project DROP UNUSED COLUMNS;
TRUNCATE TABLE project;
```

Limitations and valid-use rules

- TRUNCATE has no WHERE.
- DDL has implicit transaction effects.

- Dependencies and constraints can restrict structural changes.
- TRUNCATE is not the same transactional operation as DELETE.

Comparison with similar syntax

Feature	Best use	Key difference
DELETE	Selective/transactional row DML	TRUNCATE all-row DDL
DROP TABLE	Remove object	TRUNCATE keep object
SET UNUSED	Hide column quickly	Rename column

Exam construction pattern: Questions compare table existence and rollback behavior afterward.

12.3 Temporary and external tables

Temporary tables provide temporary rows; external tables expose external data through table metadata.

Capabilities

- Global temporary table definition persists.
- ON COMMIT DELETE ROWS uses transaction-duration rows.
- ON COMMIT PRESERVE ROWS keeps data for the session.
- External tables use directory objects, access parameters, and external files.

Syntax and example

```
CREATE GLOBAL TEMPORARY TABLE stage_ids(id NUMBER)
ON COMMIT DELETE ROWS;
CREATE TABLE ext_sales(id NUMBER,amount NUMBER)
ORGANIZATION EXTERNAL(TYPE ORACLE_LOADER
DEFAULT DIRECTORY data_dir LOCATION('sales.csv'));
```

Limitations and valid-use rules

- Temporary rows are private by session context.
- External tables are not ordinary internally stored heap data.
- External table write capabilities differ; ordinary UPDATE/DELETE expectations are inappropriate.
- Directory privileges and file access matter.

Comparison with similar syntax

Feature	Best use	Key difference
GTT	Persistent definition, temporary rows	Permanent shared rows
External table	Read external data via metadata	Regular heap storage

Exam construction pattern: The test asks what remains after COMMIT or session end.

12.4 Constraints

Constraints enforce entity, domain, and referential integrity.

Capabilities

- PRIMARY KEY is unique and non-NULL.

- UNIQUE enforces uniqueness but can be nullable.
- NOT NULL requires a value. CHECK enforces a condition.
- FOREIGN KEY requires non-NULL child values to match parent keys.

Syntax and example

```
CREATE TABLE order_line(
  order_id NUMBER,line_no NUMBER,product_id NUMBER NOT NULL,
  quantity NUMBER CHECK(quantity>0),
  CONSTRAINT order_line_pk PRIMARY KEY(order_id,line_no),
  CONSTRAINT ol_product_fk FOREIGN KEY(product_id)
  REFERENCES product(product_id));
```

Limitations and valid-use rules

- One table has one primary-key constraint, which can be composite.
- A nullable foreign key can be NULL.
- NOT NULL does not prevent duplicates.
- Composite constraints require table-level syntax.
- Parent deletion can fail when child references remain, absent allowed cascading behavior.

Comparison with similar syntax

Feature	Best use	Key difference
PRIMARY KEY	Unique row identity and non-NULL	UNIQUE nullable candidate key
FOREIGN KEY	Referential integrity	JOIN retrieval
CHECK	Row condition	DEFAULT replacement

Exam construction pattern: Choices commonly swap uniqueness, nullability, and referential rules.

Module 13 - Views

13.1 Creating and managing views

A conventional view stores a SELECT definition and exposes its result as a virtual table.

Capabilities

- Views simplify complex queries and restrict exposed columns or rows.
- Simple views may permit DML.
- WITH CHECK OPTION enforces the view predicate for changes through the view.
- WITH READ ONLY prevents DML.

Syntax and example

```
CREATE VIEW active_emp_v AS
SELECT employee_id,employee_name,department_id
FROM employees WHERE status='ACTIVE'
WITH CHECK OPTION;
```

```
CREATE VIEW dept_avg_v AS
SELECT department_id,AVG(salary) avg_salary
FROM employees GROUP BY department_id
WITH READ ONLY;
```

Limitations and valid-use rules

- DISTINCT, aggregates, GROUP BY, set operators, and many joins restrict direct DML.
- Dropping a view does not drop its base tables.
- A normal view is not a separate stored copy of base rows.
- Privilege requirements still apply.

Comparison with similar syntax

Feature	Best use	Key difference
View	Stored query definition	Synonym alternate name
WITH CHECK OPTION	Keep changed rows visible	WITH READ ONLY
Materialized view	Stored result data	Conventional virtual view

Exam construction pattern: Ask whether one view row maps unambiguously to one base row.

Module 14 - Controlling User Access

14.1 System privileges, object privileges, and roles

Privileges authorize actions; roles group privileges.

Capabilities

- System privileges permit database-level actions such as CREATE SESSION.
- Object privileges apply to named objects, such as SELECT ON HR.EMPLOYEES.
- Roles collect privileges for easier administration.
- GRANT OPTION allows onward object-privilege grants; ADMIN OPTION applies to system privileges or roles.

Syntax and example

```
GRANT CREATE SESSION TO reporter;
GRANT SELECT ON hr.employees TO reporter;
GRANT SELECT,INSERT ON hr.stage TO loader_role;
GRANT loader_role TO reporter;
```

Limitations and valid-use rules

- A synonym does not grant object privilege.
- CREATE TABLE does not permit DML on another schema table.
- Privileges obtained through roles have context limitations for stored program compilation/execution.
- Revocation can affect dependent grants.

Comparison with similar syntax

Feature	Best use	Key difference
System privilege	Database action	Specific table operation
Object privilege	Action on named object	Role bundle
Role	Collection of privileges	Synonym

Exam construction pattern: Identify the scope before selecting GRANT syntax.

Module 15 - Data Dictionary Views

15.1 USER, ALL, and DBA views

Dictionary views expose metadata about objects, columns, constraints, indexes, and privileges.

Capabilities

- USER_* describes objects owned by the current schema.
- ALL_* describes objects accessible to the user.
- DBA_* provides broad database metadata with required privilege.
- Use USER_OBJECTS, USER_TABLES, USER_TAB_COLUMNS, USER_CONSTRAINTS, USER_CONS_COLUMNS, USER_INDEXES, USER_VIEWS, USER_SEQUENCES, USER_SYNONYMS.

Syntax and example

```
SELECT object_name,object_type,status FROM user_objects;
SELECT c.constraint_name,c.constraint_type,cc.column_name
FROM user_constraints c JOIN user_cons_columns cc
ON cc.constraint_name=c.constraint_name
WHERE c.table_name='EMPLOYEES';
```

Limitations and valid-use rules

- Unquoted identifiers are normally stored uppercase in dictionary data.
- USER_* does not mean every accessible object.
- Constraint and column details may require joining dictionary views.

Comparison with similar syntax

Feature	Best use	Key difference
USER_*	Owned objects	ALL_* accessible objects
ALL_*	Accessible objects	DBA_* complete scope
DESCRIBE	Client display of structure	Dictionary query flexibility

Exam construction pattern: Questions ask which view reveals a specific metadata category.

Module 16 - Data in Different Time Zones

16.1 Current date and timestamp functions

Oracle distinguishes session-local values from database host values and distinguishes data types that retain zone information.

Capabilities

- `CURRENT_DATE` returns `DATE` in the session time zone.
- `CURRENT_TIMESTAMP` returns `TIMESTAMP WITH TIME ZONE` in the session time zone.
- `LOCALTIMESTAMP` returns `TIMESTAMP` without a zone component in session-local time.
- `SYSTIMESTAMP` reflects the database host timestamp with time-zone information.

Syntax and example

```
SELECT CURRENT_DATE, CURRENT_TIMESTAMP, LOCALTIMESTAMP, SYSTIMESTAMP
FROM dual;
```

Limitations and valid-use rules

- `DATE` has no time-zone component.
- `LOCALTIMESTAMP` has no zone component despite local session interpretation.
- Changing session time zone can change `CURRENT_*` presentation.
- Do not assume `CURRENT_TIMESTAMP` and `LOCALTIMESTAMP` have the same type.

Comparison with similar syntax

Feature	Best use	Key difference
<code>CURRENT_DATE</code>	<code>DATE</code> in session zone	<code>SYSDATE</code> host-based <code>DATE</code>
<code>CURRENT_TIMESTAMP</code>	Timestamp with zone	<code>LOCALTIMESTAMP</code> no zone
<code>SYSTIMESTAMP</code>	Host timestamp with zone	Session current timestamp

Exam construction pattern: Questions usually ask exact return data type or session-zone behavior.

16.2 TIMESTAMP and INTERVAL data types

Timestamp types model instants with varying zone semantics; interval types model durations.

Capabilities

- `TIMESTAMP` adds fractional seconds but no zone.
- `TIMESTAMP WITH TIME ZONE` stores zone information.
- `TIMESTAMP WITH LOCAL TIME ZONE` normalizes storage and presents values in session zone.
- `INTERVAL YEAR TO MONTH` models year/month durations.
- `INTERVAL DAY TO SECOND` models day/hour/minute/second durations.

Syntax and example

```
CREATE TABLE meeting_log(
  starts_at TIMESTAMP WITH TIME ZONE,
  local_recorded TIMESTAMP WITH LOCAL TIME ZONE,
  duration INTERVAL DAY TO SECOND);
SELECT TIMESTAMP '2026-09-10 09:00:00'+INTERVAL '2' HOUR FROM dual;
```

Limitations and valid-use rules

- Year-month and day-second intervals are different families.
- `DATE` is not time-zone aware.

- Local-time-zone timestamp does not preserve the original zone representation for display; it adapts to session zone.
- Daylight-saving rules can matter when regions are used.

Comparison with similar syntax

Feature	Best use	Key difference
TIMESTAMP	Fractional seconds, no zone	DATE seconds/no zone
WITH TIME ZONE	Preserve zone information	WITH LOCAL TIME ZONE session display
YEAR TO MONTH interval	Calendar months	DAY TO SECOND elapsed components

Exam construction pattern: The exam contrasts function return types and interval compatibility.

Final Practice Section - Items 26 to 55

Format: These continue the numbering after the 25 items in the first-half reviewer. Each answer is directly below the question so the section doubles as a study guide.

Practice Item 26

EMP has 4 rows and DEPT has 3 rows. A CROSS JOIN returns how many rows?

- A. 3
- B. 4
- C. 7
- D. 12

Correct answer: D

Why

Every EMP row pairs with every DEPT row.

Alternative analysis

- A. Counts one source only.
- B. Counts the other source only.
- C. Adds instead of multiplies.
- D. Correct: 4×3 .

Rule to remember: Cartesian row counts multiply.

Practice Item 27

Which join preserves every employee when some employees have no department?

- A. EMP INNER JOIN DEPT
- B. EMP LEFT JOIN DEPT
- C. EMP CROSS JOIN DEPT
- D. DEPT LEFT JOIN EMP only

Correct answer: B

Why

EMP is the preserved left source.

Alternative analysis

- A. Drops unmatched employees.
- B. Correct.
- C. Creates all combinations.
- D. Preserves departments, not necessarily all employees.

Rule to remember: Preserved side matters.

Practice Item 28

Why can WHERE d.status='OPEN' after a LEFT JOIN remove unmatched right rows?

- A. Unmatched d.status is NULL and fails the WHERE predicate.
- B. LEFT JOIN becomes CROSS JOIN automatically.
- C. WHERE creates a primary key.
- D. It cannot remove them.

Correct answer: A

Why

Right-side columns are NULL for unmatched rows, so the WHERE condition is not TRUE.

Alternative analysis

- A. Correct.
- B. No Cartesian conversion.
- C. Queries do not create constraints.
- D. It can change effective preservation.

Rule to remember: Filter placement can alter outer-join results.

Practice Item 29

A scalar subquery used with = returns two rows. What happens?

- A. Oracle chooses the first row.
- B. A single-row subquery error occurs.
- C. The values are concatenated.
- D. The query becomes IN automatically.

Correct answer: B

Why

A scalar comparison needs at most one row.

Alternative analysis

- A. No arbitrary choice.
- B. Correct.
- C. No concatenation.
- D. No automatic operator conversion.

Rule to remember: Match operator to cardinality.

Practice Item 30

Which statement about NOT IN is true when the subquery may return NULL?

- A. NULL is ignored automatically.
- B. The predicate can become UNKNOWN and return no rows.
- C. It always equals NOT EXISTS.
- D. NULL becomes zero.

Correct answer: B

Why

A comparison against NULL is UNKNOWN.

Alternative analysis

- A. Incorrect.
- B. Correct.
- C. NULL behavior differs.
- D. No automatic conversion.

Rule to remember: Prefer careful NULL filtering or suitable NOT EXISTS.

Practice Item 31

Which operator returns rows common to both queries?

- A. UNION ALL
- B. MINUS
- C. INTERSECT
- D. CROSS JOIN

Correct answer: C

Why

INTERSECT keeps common complete rows.

Alternative analysis

- A. Combines all.
- B. First-only.
- C. Correct.
- D. Pairs rows horizontally.

Rule to remember: Set operation names are literal clues.

Practice Item 32

Which statement about UNION ALL is true?

- A. It removes duplicates.
- B. It retains duplicates.
- C. It permits different column counts.
- D. It sorts automatically.

Correct answer: B

Why

UNION ALL combines rows without duplicate elimination.

Alternative analysis

- A. UNION removes duplicates.
- B. Correct.
- C. Column counts must match.
- D. Use ORDER BY.

Rule to remember: ALL keeps duplicates.

Practice Item 33

Where is ORDER BY placed in an ordinary compound set query?

- A. After every component
- B. Once at the end
- C. Before the first SELECT
- D. Inside UNION keyword

Correct answer: B

Why

The final ORDER BY sorts the combined result.

Alternative analysis

- A. Not ordinary syntax.
- B. Correct.
- C. Invalid placement.
- D. No such syntax.

Rule to remember: Sort the final set result.

Practice Item 34

DELETE FROM employees; does what?

- A. Fails without WHERE
- B. Targets all rows but keeps the table definition
- C. Drops the table
- D. Deletes only NULL rows

Correct answer: B

Why

WHERE is optional, so all rows qualify.

Alternative analysis

- A. Valid syntax.
- B. Correct.
- C. DROP removes the object.
- D. No NULL filter exists.

Rule to remember: Always verify UPDATE/DELETE scope.

Practice Item 35

INSERT, SAVEPOINT S, UPDATE, ROLLBACK TO S, COMMIT leaves what?

- A. Nothing
- B. The insert committed; update undone
- C. Update committed; insert undone
- D. Both undone

Correct answer: B

Why

Only work after the savepoint is reversed.

Alternative analysis

- A. Insert precedes savepoint.
- B. Correct.
- C. Reverse chronology.
- D. COMMIT saves surviving insert.

Rule to remember: Draw a transaction timeline.

Practice Item 36

INSERT FIRST with two true WHEN branches for one source row executes:

- A. Both branches
- B. Only the first matching branch
- C. Neither branch
- D. A UNION

Correct answer: B

Why

INSERT FIRST stops branch evaluation after the first match for that source row.

Alternative analysis

- A. That describes INSERT ALL.
- B. Correct.
- C. A branch is true.
- D. Unrelated.

Rule to remember: FIRST versus ALL is decisive.

Practice Item 37

MERGE WHEN MATCHED commonly performs:

- A. UPDATE
- B. CREATE TABLE
- C. GRANT
- D. COMMIT automatically

Correct answer: A

Why

Matched target rows can be updated.

Alternative analysis

- A. Correct.
- B. DDL.
- C. Privilege command.
- D. MERGE remains transactional.

Rule to remember: Matched and not matched drive branches.

Practice Item 38

Which statement about indexes is true?

- A. Every available index must be used.
- B. Indexes can improve access but add storage and DML maintenance.
- C. Every index guarantees uniqueness.
- D. Dropping an index drops the table.

Correct answer: B

Why

Indexes are optional optimizer-selected access structures.

Alternative analysis

- A. Full scan may be cheaper.
- B. Correct.
- C. Nonunique indexes exist.
- D. Table remains.

Rule to remember: Benefit versus maintenance cost.

Practice Item 39

A synonym without SELECT privilege allows a user to:

- A. Query the target anyway
- B. Resolve the alternate name but not bypass authorization
- C. Become object owner
- D. Create an index automatically

Correct answer: B

Why

Naming and authorization are separate.

Alternative analysis

- A. Privilege still required.
- B. Correct.
- C. Ownership is unchanged.
- D. No automatic index.

Rule to remember: Synonym is not permission.

Practice Item 40

CURRVAL before NEXTVAL in a new session is:

- A. Always 1
- B. Not yet defined for the session
- C. The sequence maximum
- D. NULL silently

Correct answer: B

Why

NEXTVAL must establish the session current value.

Alternative analysis

- A. START WITH may differ.
- B. Correct.
- C. Not maximum.
- D. It is not silently NULL.

Rule to remember: Sequence session state matters.

Practice Item 41

A rolled-back INSERT using NEXTVAL causes the sequence value to:

- A. Be restored
- B. Remain consumed, allowing a gap
- C. Become a foreign key
- D. Commit the insert

Correct answer: B

Why

Sequence generation is not transactional rollback of the number.

Alternative analysis

- A. Incorrect.
- B. Correct.
- C. Unrelated.
- D. The row can be rolled back.

Rule to remember: Sequences are not gapless.

Practice Item 42

TRUNCATE TABLE employees:

- A. Can use WHERE
- B. Removes all rows and retains the table
- C. Removes the table definition
- D. Is ordinary rollback-able DELETE

Correct answer: B

Why

TRUNCATE empties the table as DDL.

Alternative analysis

- A. No WHERE.
- B. Correct.
- C. DROP removes definition.
- D. Different transaction behavior.

Rule to remember: Separate DELETE, TRUNCATE, DROP.

Practice Item 43

ALTER TABLE t SET UNUSED(c) primarily:

- A. Renames c
- B. Makes c inaccessible for later physical cleanup
- C. Indexes c
- D. Sets every c value to NULL

Correct answer: B

Why

UNUSED removes logical accessibility quickly.

Alternative analysis

- A. No rename.
- B. Correct.
- C. No index.
- D. Not row DML.

Rule to remember: UNUSED is a structural operation.

Practice Item 44

ON COMMIT DELETE ROWS means GTT rows:

- A.** Are cleared at commit
- B.** Are permanent
- C.** Drop the table at commit
- D.** Become public

Correct answer: A

Why

Transaction-duration rows are cleared.

Alternative analysis

- A.** Correct.
- B.** Rows are temporary.
- C.** Definition remains.
- D.** Session isolation remains.

Rule to remember: Definition lifetime and row lifetime differ.

Practice Item 45

Which statement about a nullable foreign key is true?

- A.** It may contain NULL.
- B.** It must always match zero.
- C.** It prevents duplicates automatically.
- D.** It is the primary key.

Correct answer: A

Why

NULL asserts no parent key unless NOT NULL also applies.

Alternative analysis

- A.** Correct.
- B.** Zero needs a parent zero.
- C.** Foreign key does not guarantee uniqueness.
- D.** Different constraint.

Rule to remember: Referential integrity applies to non-NULL child keys.

Practice Item 46

A composite primary key (order_id,line_no) permits:

- A. NULL line_no
- B. The same order_id with different line_no values
- C. Duplicate pairs
- D. A second independent primary key

Correct answer: B

Why

Uniqueness applies to the pair; each PK component is non-NULL.

Alternative analysis

- A. PK component cannot be NULL.
- B. Correct.
- C. Duplicate pair violates.
- D. One PK constraint.

Rule to remember: Composite means combination uniqueness.

Practice Item 47

Which view is normally not directly updatable?

- A. Simple one-table projection
- B. GROUP BY aggregate view
- C. Single-table view with key
- D. Simple filtered view

Correct answer: B

Why

A summary row does not map unambiguously to one base row.

Alternative analysis

- A. Often updatable.
- B. Correct.
- C. Often updatable.
- D. Often updatable, subject to rules.

Rule to remember: View DML requires clear row mapping.

Practice Item 48

WITH CHECK OPTION prevents:

- A. SELECT
- B. A change through the view that makes the row no longer satisfy the view predicate
- C. Creating the view
- D. ORDER BY

Correct answer: B

Why

Changed rows must remain visible through the view.

Alternative analysis

- A. SELECT remains.
- B. Correct.
- C. It is part of creation.
- D. Unrelated.

Rule to remember: Check option preserves predicate visibility.

Practice Item 49

CREATE SESSION is a:

- A. System privilege
- B. Object privilege
- C. Synonym
- D. Constraint

Correct answer: A

Why

It grants a database-level action.

Alternative analysis

- A. Correct.
- B. Object privileges name an object.
- C. Alias object.
- D. Integrity rule.

Rule to remember: Identify privilege scope.

Practice Item 50

SELECT ON hr.employees is a:

- A. System privilege
- B. Object privilege
- C. Role only
- D. Data type

Correct answer: B

Why

It applies to a named object.

Alternative analysis

- A. Not database-wide action.
- B. Correct.
- C. It can be placed in a role but is itself an object privilege.
- D. Unrelated.

Rule to remember: ON object signals object privilege.

Practice Item 51

Which prefix describes objects owned by the current user?

- A. USER_
- B. ALL_ only
- C. DBA_ only
- D. SYSFILE_

Correct answer: A

Why

USER_* views focus on current-schema ownership.

Alternative analysis

- A. Correct.
- B. ALL_* is accessible scope.
- C. DBA_* broad scope.
- D. Not the standard family.

Rule to remember: USER, ALL, DBA scopes differ.

Practice Item 52

Unquoted object names are commonly searched in dictionary views as:

- A. Lowercase
- B. Uppercase
- C. Encrypted
- D. Numeric only

Correct answer: B

Why

Oracle normally stores unquoted identifiers in uppercase metadata form.

Alternative analysis

- A. Not the default stored form.
- B. Correct.
- C. No.
- D. No.

Rule to remember: Dictionary filters commonly use uppercase names.

Practice Item 53

CURRENT_TIMESTAMP returns:

- A. DATE without zone
- B. TIMESTAMP WITH TIME ZONE in the session time zone
- C. INTERVAL
- D. NUMBER

Correct answer: B

Why

The function returns a time-zone-aware timestamp for the session zone.

Alternative analysis

- A. CURRENT_DATE returns DATE.
- B. Correct.
- C. Not a duration.
- D. Not numeric.

Rule to remember: Know exact return data types.

Practice Item 54

LOCALTIMESTAMP returns:

- A.** TIMESTAMP without a time-zone component
- B.** TIMESTAMP WITH TIME ZONE
- C.** DATE only
- D.** INTERVAL YEAR TO MONTH

Correct answer: A

Why

It reflects session local time but the result type has no zone component.

Alternative analysis

- A.** Correct.
- B.** CURRENT_TIMESTAMP includes zone.
- C.** It is TIMESTAMP.
- D.** Not an interval.

Rule to remember: Local does not mean zone is included in the type.

Practice Item 55

Which interval represents 2 years and 3 months?

- A.** INTERVAL YEAR TO MONTH
- B.** INTERVAL DAY TO SECOND
- C.** DATE
- D.** VARCHAR2 only

Correct answer: A

Why

Year/month durations use the YEAR TO MONTH family.

Alternative analysis

- A.** Correct.
- B.** Elapsed days and smaller units.
- C.** An instant, not duration family.
- D.** Text is not the intended interval type.

Rule to remember: Do not mix the two interval families.

Final-Half Comparison Matrix

Often confused	Correct distinction	Exam trap
INNER vs OUTER JOIN	Matches only vs preservation of unmatched rows	WHERE predicate removes preserved NULL row
SELF JOIN vs two tables	One table, two aliases/roles	Claiming two physical copies required
IN vs = subquery	Set vs scalar	Multiple-row error with =
NOT IN vs NOT EXISTS	NULL-sensitive set comparison vs existence logic	NULL causes UNKNOWN
UNION vs UNION ALL	Duplicate removal vs retention	Claiming ALL removes duplicates
DELETE vs TRUNCATE vs DROP	Row DML vs all-row DDL vs object removal	Rollback and table-existence confusion
INSERT ALL vs INSERT FIRST	All matching branches vs first match	Counting target rows incorrectly
Index vs constraint	Access structure vs integrity rule	Treating every index as unique
Synonym vs privilege	Name resolution vs authorization	Assuming synonym grants SELECT
NEXTVAL vs CURRVAL	Advance vs current session value	CURRVAL before NEXTVAL
PRIMARY KEY vs UNIQUE	One identifying non-NULL key vs nullable candidate uniqueness	Multiple PK constraints
View vs synonym	Stored query vs alternate object name	Assuming either copies data
System vs object privilege	Database action vs named-object action	CREATE SESSION vs SELECT ON
USER vs ALL vs DBA	Owned vs accessible vs broad database scope	Using USER_* for others objects
CURRENT_TIMESTAMP vs LOCALTIMESTAMP	With zone vs without zone component	Assuming same return type
YEAR TO MONTH vs DAY TO SECOND	Calendar units vs elapsed day/time units	Mixing interval families

Final Exam Readiness Checklist

- Calculate join row counts and preserved rows manually.
- Choose scalar versus set subquery operators and account for NULL.
- Validate set-query column counts, type positions, duplicate behavior, and final order.
- Trace transactions chronologically, including savepoints and DDL boundaries.
- Explain the benefits and limitations of indexes, synonyms, and sequences.
- Differentiate DELETE, TRUNCATE, DROP, UNUSED, temporary tables, and external tables.

- Apply primary, unique, foreign-key, check, and not-null rules to sample inserts.
- Determine whether a view is updatable and what CHECK OPTION or READ ONLY changes.
- Select the correct privilege category and dictionary-view prefix.
- Identify time-zone function return types and valid interval families.

Completion: When this final-half reviewer is combined with <File>Oracle SQL First-Half Exam Reviewer</File>, every topic visible in the supplied exam outline is covered.

Sources and Scope

Content is aligned to the user-supplied exam outline and Oracle SQL terminology. Technical rules were checked against Oracle Database SQL Language Reference material for joins, queries and subqueries, set operators, DML/DDDL objects, and datetime data types.

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Joins.html>

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/SQL-Queries-and-Subqueries.html>

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/Using-Subqueries.html>

<https://docs.oracle.com/en/database/oracle/oracle-database/19/sqlrf/index.html>